Listing 10.7 Injecting strongly typed options into a handler using `IOptions<T>`

```
app.MapGet("/display-settings",
    (IOptions<AppDisplaySettings> options) =>          ❶
{
    AppDisplaySettings settings = options.Value;       ❷
    string title = settings.Title;                     ❸
    bool showCopyright = settings.ShowCopyright;       ❹

    return new { title, showCopyright };
});
```

❶ You can inject a strongly typed options class using the IOptions<> wrapper interface.

❷ The Value property exposes the POCO settings object.

❸ The settings object contains properties that are bound to configuration values at runtime.

❹ The binder can also convert string values directly to built-in types.

The ASP.NET Core configuration system includes a *binder*, which can take a collection of configuration values and bind them to a strongly typed object, called an *options class*. This binding is similar to the concept of JSON deserialization for creating types from chapter 6 and the model binding used by Model-View-Controller (MVC) and Razor Pages, which you'll learn about in part 3.

Section 10.3.1 shows how to set up the binding of configuration values to a POCO options class, and section 10.3.2 shows how to make sure that it reloads when the underlying configuration values change. We'll look at the different sorts of objects you can bind in section 10.3.3.

10.3.1 Introducing the IOptions interface

ASP.NET Core introduced strongly typed settings as a way of letting configuration code adhere to the single-responsibility principle (SRP) and to allow the injection of configuration classes as explicit dependencies. Such settings also make testing easier; instead of having to create an instance of `IConfiguration` to test a service, you can create an instance of the POCO options class.